

Digital Campus

*An Intelligent, Source-Grounded Assistant
for the Faculté des Sciences de Bizerte*

Summer Internship Report — 2026

Project codename: **FSB Nexus**

Team member	Primary role
Aymen Jeddou	RAG intelligent system (Role 1) • Scrum Master • branch integration
Iheb Bouali	Knowledge base • frontend enhancement (better_front)
Abderrahim Neji	Backend development • Google Classroom integration (Role 2)
Mohamed Amine Trabelsi	Frontend design, then security & platform hardening (Role 4)

Academic year 2025–2026

Supervisor: _____ • Internship host: _____
Internship period: _____

Contents

1. Introduction and Context	3
2. Project Overview	3
2.1 Feature summary	3
3. Team Organization and Work Distribution	4
3.1 Responsibility matrix	4
3.2 Per-member contributions	4
3.3 Branch and integration workflow	5
4. Technical Architecture	5
4.1 Frontend (Next.js) — Amin, then Iheb	5
4.2 Backend (FastAPI) — Abderrahim, hardened by Amin	5
4.3 AI / RAG pipeline — Aymen	5
4.4 Knowledge base — Iheb	5
4.5 Courses and Google Classroom — Abderrahim	5
5. The RAG Intelligent System in Depth	6
5.1 Evaluation harness	6
5.2 Key improvements	6
5.3 Measured results	6
6. Final Production-Readiness Sprint	7
7. Testing and Quality Assurance	7
8. Deployment	7
9. Challenges and Lessons Learned	7
10. Conclusion and Future Work	8
Appendix A. Technology Stack	8
Appendix B. Repository, Branches and Pull Requests	8

1. Introduction and Context

Students at the Faculté des Sciences de Bizerte (FSB) face a familiar problem: the information they need — admission conditions, available licences and masters, course prerequisites, the academic calendar, administrative procedures — is scattered across PDFs, notice boards and word of mouth. General-purpose chatbots answer confidently but invent details, which is worse than no answer at all in an academic setting.

Objective. During the summer 2026 internship, our four-person team built **Digital Campus** (codename *FSB Nexus*): a web platform whose centrepiece is an **agentic Retrieval-Augmented Generation (RAG)** assistant that answers questions *only* from official FSB documents and **cites its sources** on every answer. If the documents do not support an answer, the assistant says so rather than guessing.

Scope of this report. This is a team report. It describes the whole platform, explains how the four of us divided the work, and then goes deeper on each technical area — with a dedicated chapter on the RAG intelligent system and its measured results. All work described here currently lives on the **develop** branch and is pending a final merge to **main**.

At a glance. Four layers (frontend, backend, AI/RAG, knowledge base); a multilingual French/Arabic/English assistant; streamed, cited answers; a courses module with Google Classroom import; and a final hardening sprint that added onboarding, server-side access control, rate limiting, database migrations and continuous integration.

2. Project Overview

Digital Campus is composed of four cooperating layers. The browser talks to the backend; the backend forwards questions to the AI/RAG pipeline; the pipeline retrieves passages from a PostgreSQL database (with the **pgvector** extension) built from cleaned FSB documents, and uses the Mistral large language model to write a grounded, cited answer.

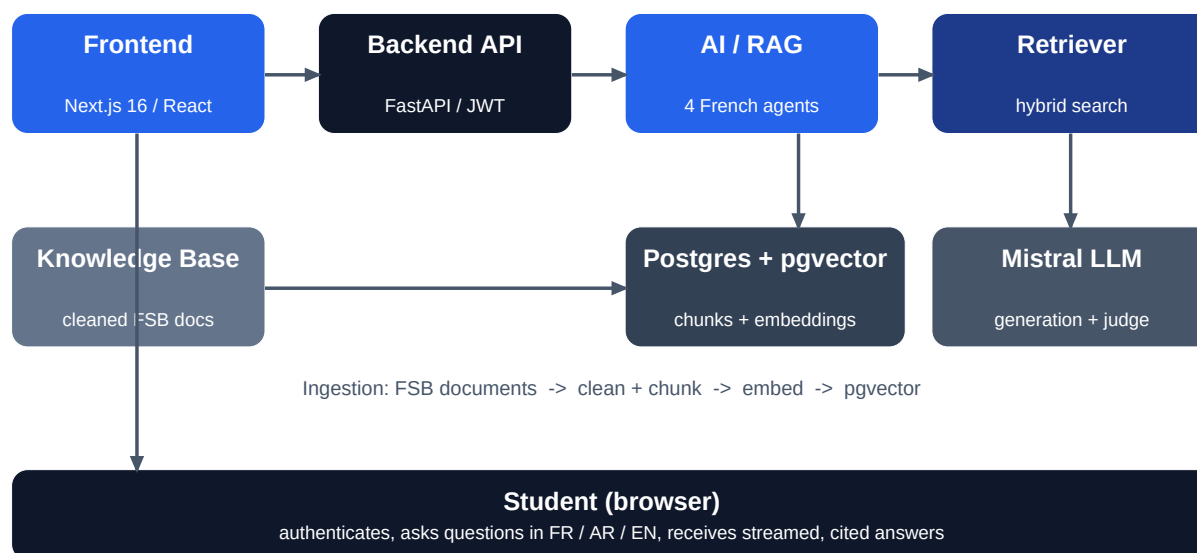


Figure 1 — High-level architecture of the Digital Campus platform.

2.1 Feature summary

- **Authentication and onboarding** — register, email verification, login (JWT), and a guided onboarding flow that sets the student profile.
- **Chat assistant** — streamed, token-by-token answers with inline citations to the source document and page; conversation history; and thumbs up/down feedback.

- **Dashboard** — the student's real profile, courses, materials and recent conversations.
- **Courses and Google Classroom** — add courses, upload materials (PDF/DOCX/TXT), or import them read-only from Google Classroom; imported content is scoped privately to that student.
- **Knowledge base** — official FSB documents cleaned, chunked, catalogued and embedded for retrieval.

3. Team Organization and Work Distribution

We worked in an **Agile / Scrum** style across two main sprints plus a final integration and hardening phase. **Aymen Jeddou** acted as **Scrum Master**, coordinating the backlog, and — because the four areas advanced in parallel — also owned the **integration** of everyone's branches into a single working **develop** branch. The project blueprint (`docs/FSB_Nexus_Blueprint.tex`) defined four roles, each owning a complete, demonstrable outcome; the team then assigned members to roles.

3.1 Responsibility matrix

Blueprint role	Owner(s)	Main deliverable
Role 1 — Conversational Intelligence & Retrieval	Aymen Jeddou	The 4-agent RAG pipeline, retrieval quality, and the evaluation benchmark
Knowledge Base	Iheb Bouali	Harvested, cleaned, chunked and catalogued FSB documents
Role 2 — Courses & Integrations	Abderrahim Neji	Courses module + Google Classroom OAuth import + file upload ingestion
Role 3 — Frontend & Experience	Mohamed Amine Trabelsi (design), Iheb Bouali (enhancement)	Next.js auth, dashboard and chat UI; refreshed design in <code>better_front</code>
Role 4 — Backend, Security & Platform	Abderrahim Neji (core API), Mohamed Amine Trabelsi (security)	FastAPI auth/profile/chat/documents, streaming endpoint, hardening
Scrum & Integration	Aymen Jeddou	Backlog coordination and merging all branches into <code>develop</code>

3.2 Per-member contributions

Aymen Jeddou — RAG intelligent system, Scrum Master, integration. Designed and built the agentic RAG pipeline (pull requests #7, #11), integrated it end-to-end with the backend and ingestion (#16), and led a second data/quality sprint that fixed retrieval and over-refusal and added an evaluation harness (#19). As Scrum Master he coordinated the team and personally merged the backend, courses and frontend branches into **develop**, resolving conflicts and closing quality gaps found during integration. Chapters 5 and 6 detail this work.

Iheb Bouali — knowledge base and frontend enhancement. Built the knowledge base: harvested official FSB documents, then cleaned, chunked and catalogued them into the pipeline that feeds retrieval (#9, #14). He also enhanced the frontend on the `better_front` branch, refining the design and user experience. (Some of this work appears under a second account, `o-MrNobody-o`.)

Abderrahim Neji — backend and integrations. Implemented the core backend (authentication, profile, chat and documents endpoints, and all associated tests; #10), then built the Courses & Integrations module (#20): a manual course/material path plus a read-only Google Classroom import using OAuth 2.0 with encrypted token storage and file-upload ingestion.

Mohamed Amine Trabelsi — frontend design, then security and platform. Delivered the first frontend sprint (authentication, dashboard and chat UI, light/dark theme; #12, #13), then shifted to Role 4 and implemented backend security and platform upgrades — including the streaming response endpoint, a feedback model, and hardening measures (#18).

3.3 Branch and integration workflow

Each role worked on its own feature branch and opened a pull request. Aymen reviewed and merged them into **develop**. Integration was not a rubber stamp: for example, the streaming endpoint initially bypassed the RAG safety path (groundedness grading, intent routing and conversation memory), which was caught and rewired during integration so that streamed answers carry the same protections as non-streamed ones.

Pull request	Author	Area	Merged into
#7, #11	Aymen	RAG generation pipeline (agents, prompts, LLM)	develop
#9, #14	Iheb	Knowledge base import / clean / chunk	develop
#10	Abderrahim	Backend: auth, profile, chat, documents	develop
#12, #13	Amin	Frontend Sprint 1: auth, dashboard, chat UI	develop
#16, #17	Aymen	End-to-end RAG integration; Sprint-1 consolidation	develop / main
#18	Amin	Role 4: security, streaming endpoint, platform	develop
#19	Aymen	KB sprint 2: retrieval + eval harness	develop
#20	Abderrahim	Courses & Google Classroom integration	develop
#21 (better_front)	Iheb / Aymen	Frontend redesign + streaming/courses UI	develop

4. Technical Architecture

4.1 Frontend (Next.js) — Amin, then Iheb

The frontend is a **Next.js 16** application (App Router, Turbopack) styled with Tailwind CSS and animated with Framer Motion. It provides authentication screens, a dashboard, a courses screen, and the chat interface. The chat consumes the backend's **Server-Sent Events (SSE)** stream to render answers token-by-token, showing citation chips beneath each reply. Light and dark themes are supported through semantic design tokens.

4.2 Backend (FastAPI) — Abderrahim, hardened by Amin

The backend is a **FastAPI** REST API using JWT authentication. It exposes authentication (register / verify / login), profile and onboarding, chat (send, stream, sessions, messages, feedback), documents, and courses endpoints. Passwords are hashed; protected routes require a bearer token. Later hardening added a streaming endpoint, a login rate limiter backed by Redis, and structured logging.

4.3 AI / RAG pipeline — Aymen

The heart of the platform is a set of **four specialised French-language agents** (Orientation, Academic, Administrative, Learning). A question is routed to the right agent, which runs the pipeline: **retrieve** candidate passages, **filter** them, **generate** a grounded answer with the LLM, **format citations** as **[document, p.X]**, and **grade groundedness** (a second LLM check that the answer is supported by the sources). The LLM client is provider-agnostic (Mistral by default). Chapter 5 details the retrieval and quality work.

4.4 Knowledge base — Iheb

Official FSB documents are harvested and passed through a cleaning and chunking pipeline that splits them into passages, attaches metadata (source document, page, category), and embeds each chunk with a multilingual sentence-transformer model (768-dimensional vectors) into the **document_chunks** table. Global knowledge-base chunks are visible to everyone; course-material chunks are scoped to a single student.

4.5 Courses and Google Classroom — Abderrahim

Students can add a course and attach materials manually (typed text or an uploaded PDF/DOCX/TXT), or connect Google Classroom to import courses and materials **read-only**. The integration uses the OAuth 2.0 authorization-code flow with five read-only Classroom/Drive scopes; the callback authenticates the student via a signed **state** token, and access/refresh tokens are encrypted at rest. Imported materials are ingested into the same retrieval store, scoped to that student.

5. The RAG Intelligent System in Depth

The first working assistant was *too* cautious: it refused almost every question, and when it did answer, retrieval often failed to surface passages that were actually present in the knowledge base. This chapter summarises how the system was measured and improved.

5.1 Evaluation harness

To improve retrieval without guessing, we built an **evaluation harness** with a labelled set of questions (**golden.jsonl**) and a larger 100-question set. It measures retrieval quality (**hit@5**, **MRR**, **recall@10**) and, with the real LLM, end-to-end quality (**answer rate**, **answer-with-citation rate**, and **false-answer rate** — how often the assistant invents an answer to an impossible question). This benchmark served as the shared quality bar for the whole team.

5.2 Key improvements

- **Index / probe fix** — the vector index scanned only one cluster, missing exact matches; scanning every cluster restored recall at negligible cost.
- **Softened prompts + intent routing** — the agents stopped refusing partial-but-useful answers, and greetings/meta questions are answered directly instead of going through retrieval.
- **Licence and master fact cards** — curated, well-formed entries so programme questions reliably retrieve the right card at rank 1.
- **Hybrid retrieval** — combining semantic (vector) search with PostgreSQL full-text search via reciprocal-rank fusion, so both meaning and exact keywords are found.
- **Conversation memory** — recent turns are fed back so short follow-ups (“and for chemistry?”) still retrieve the right documents.
- **Groundedness grader** — a second LLM check blocks answers the sources do not support.

5.3 Measured results

Each change was measured against the benchmark. The table shows the progression from the strict starting point to the final integrated system.

Configuration	hit@5	MRR	answer rate	false answer
Starting point (strict prompt, single-probe index)	0.50	0.395	0.60	0.00
+ index fix	0.80	0.612	0.60	0.00
+ softened prompts & intent routing	0.80	0.612	0.70	0.00
+ licence fact cards	0.80	0.678	—	—
+ hybrid retrieval	0.90	0.750	0.833	0.00
+ master cards & memory (final)	0.90	0.776	0.833	0.00–0.09

Retrieval hit@5 rose from **0.50 to 0.90** and recall@10 reached **0.967**; ranking quality (MRR) nearly doubled to **0.776**. Crucially, **every answered question carries citations** (citation rate equals answer rate), so the gain is real sourced answers, not vague hedging. In the final production sprint (Chapter 6) a routing fix took the answer rate to effectively **1.0** on the reliability set.

6. Final Production-Readiness Sprint

A closing sprint made the platform demonstrable and safe to pilot. The changes below were implemented and merged to `develop`.

- **Streaming that actually renders** — the chat had stalled on a loading state because the browser parsed Server-Sent Events with the wrong delimiter; fixing the parser made answers stream live with their citations.
- **Latency** — warming the embedding model and language-model clients at startup, plus caching, cut cold responses dramatically and gave a fast first token.
- **Real dashboard and identity** — the dashboard, header and sidebar now show the signed-in student's real profile, courses and conversations instead of placeholder mock data, with a logout action.
- **Onboarding and the Academic agent** — a new onboarding endpoint is the only way to become an *enrolled* student, which unlocked the previously unreachable Academic agent; a multi-run probe confirmed it answers (0 refusals over 9 runs) rather than over-refusing.
- **Security hardening** — a server-side route guard redirects unauthenticated requests before any page is sent; the login rate limiter was switched on with Redis (a burst of bad logins now returns HTTP 429); and structured, timestamped logging was configured.
- **Migrations and CI** — the database schema is now versioned with Alembic, and a GitHub Actions pipeline runs the AI suite (43 tests) and the backend suite (26 tests) on every push and pull request.

Alongside the platform, two companion documents were produced: a changes & deployment guide, and this report. The deployment guide covers local setup, the Google Cloud setup for Google Classroom, and a step-by-step Google Cloud deployment.

7. Testing and Quality Assurance

- **AI / RAG unit tests (43)** — cover retrieval delegation, citation formatting, the groundedness grader and the pipeline, with the language model mocked (no external calls).
- **Backend API tests (26)** — exercise the real register / verify / login flow, protected routes, chat persistence, document upload and the courses/Classroom endpoints.
- **Evaluation harness** — the retrieval and end-to-end metrics of Chapter 5, used as the shared quality benchmark.
- **Continuous integration** — both suites run automatically on GitHub Actions against a PostgreSQL+pgvector service after applying the Alembic migration.

8. Deployment

Locally, the stack runs with Docker (PostgreSQL + Redis), a shared Python environment for the backend and AI packages, and a Node environment for the frontend; the schema is created with `alembic upgrade head` and the knowledge base is ingested once.

On Google Cloud, the recommended path is Cloud SQL for PostgreSQL (with the `vector` extension), Memorystore for Redis, and two Cloud Run services (backend and frontend), with secrets in Secret Manager. The Google Classroom integration requires enabling the Classroom and Drive APIs, configuring the OAuth consent screen with the five read-only scopes, and registering the backend's callback URL. Full commands are in the companion deployment guide.

9. Challenges and Lessons Learned

- **Parallel work needs firm interfaces.** Agreeing the chat/streaming contract up front let the four roles progress independently — but integration still surfaced gaps (the streaming path that skipped the safety checks) that only a careful merge caught.

- **Measure before optimising.** The retrieval gains came from the evaluation harness, not intuition; the numbers showed which change actually helped.
- **Refusing well is a feature.** An assistant that says “I don't have that in the documents” is more trustworthy than one that guesses; balancing this against over-refusal was a central theme.
- **Latency is part of correctness of experience.** A correct answer that never renders, or takes 30 seconds, is a failure to the user; warmup and streaming addressed this.

10. Conclusion and Future Work

Over the summer 2026 internship the team delivered a working, source-grounded academic assistant for the FSB: a polished frontend, a secure backend, a measured and reliable RAG pipeline, a real knowledge base, and a courses integration — unified on the **develop** branch and ready for a final review before merging to **main**.

Future work (deferred for a later hardening pass): migrate the session to httpOnly cookies and add refresh tokens and password reset; wire real email (SMTP) for verification; complete Google's OAuth verification for public Classroom use; and broaden the evaluation set to cover course-content questions.

Appendix A. Technology Stack

Layer	Technologies
Frontend	Next.js 16 (App Router), React, TypeScript, Tailwind CSS, Framer Motion
Backend	FastAPI, SQLAlchemy, JWT, Alembic, Redis (rate limiting)
AI / RAG	Python, Mistral LLM (provider-agnostic client), sentence-transformers embeddings
Data	PostgreSQL + pgvector; hybrid vector + full-text retrieval
Integrations	Google Classroom & Drive (OAuth 2.0, read-only)
Platform	Docker Compose, GitHub Actions CI, Google Cloud (Cloud Run, Cloud SQL, Memorystore)

Appendix B. Repository, Branches and Pull Requests

All work is on the **develop** branch of the team's GitHub repository. Key feature branches: **feature/rag-pipeline**, **feature/kb-sprint2**, **feature/backend**, **amine-backend**, **courses-and-integrations**, **feature/frontend**, **better_front**. Pull requests #7–#21 record the detailed history and reviews.